

# Options Pattern in Your Project

**Answer: YES, you ARE using the Options pattern in your project.**

## Part 1: What is the Options Pattern?

The Options pattern is a way to strongly-type your configuration from appsettings.json into C# classes.

### Without Options Pattern:

```
// Reading configuration directly (not type-safe)
var connectionString =
builder.Configuration["DatabaseSettings:ConnectionString"];
var databaseName =
builder.Configuration["DatabaseSettings:DatabaseName"];
// Easy to make typos, no IntelliSense, no compile-time checking
```

### With Options Pattern:

```
// Strongly-typed configuration
var settings = options.Value; // IntelliSense, compile-time checking
var connectionString = settings.ConnectionString;
var databaseName = settings.DatabaseName;
```

## Part 2: Where You Use It in Your Project

### Catalog Service

**Settings Class:** Catalog.Infrastructure/Settings/DatabaseSettings.cs

```
public class DatabaseSettings
{
    public string ConnectionString { get; set; }
    public string DatabaseName { get; set; }
}
```

```

    public string BrandCollectionName { get; set; }
    public string TypeCollectionName { get; set; }
    public string ProductCollectionName { get; set; }
}

```

**Configuration in Program.cs** (Lines 46-47):

```

builder.Services.Configure<DatabaseSettings>(
    builder.Configuration.GetSection("DatabaseSettings"));

```

**Usage in Repository** (ProductRepository.cs Lines 16-23):

```

public ProductRepository(IOptions<DatabaseSettings> options)
{
    var settings = options.Value;
    var client = new MongoClient(settings.ConnectionString);
    var db = client.GetDatabase(settings.DatabaseName);
    _brands =
db.GetCollection<ProductBrand>(settings.BrandCollectionName);
    _types =
db.GetCollection<ProductType>(settings.TypeCollectionName);
    _products =
db.GetCollection<Product>(settings.ProductCollectionName);
}

```

## Basket Service

**Settings Classes:**

- Basket.Infrastructure/Settings/CacheSettings.cs
- Basket.Application/Settings/GrpcSettings.cs

**CacheSettings.cs:**

```

public class CacheSettings
{
    public string ConnectionString { get; set; } = string.Empty;
}

```

```
}
```

#### **GrpcSettings.cs:**

```
public class GrpcSettings
{
    public string DiscountUrl { get; set; } = string.Empty;
}
```

#### **Configuration in Program.cs (Lines 42-46):**

```
builder.Services.Configure<CacheSettings>(
    builder.Configuration.GetSection("CacheSettings"));

builder.Services.Configure<GrpcSettings>(
    builder.Configuration.GetSection("GrpcSettings"));
```

#### **Usage for gRPC Client (Lines 49-54):**

```
builder.Services.AddGrpcClient<DiscountProtoService.DiscountProtoServiceClient>(
    (sp, cfg) =>
    {
        var grpcSetting =
            sp.GetRequiredService<IOptions<GrpcSettings>>().Value;
        cfg.Address = new Uri(grpcSetting.DiscountUrl);
    });
```

## **Discount Service**

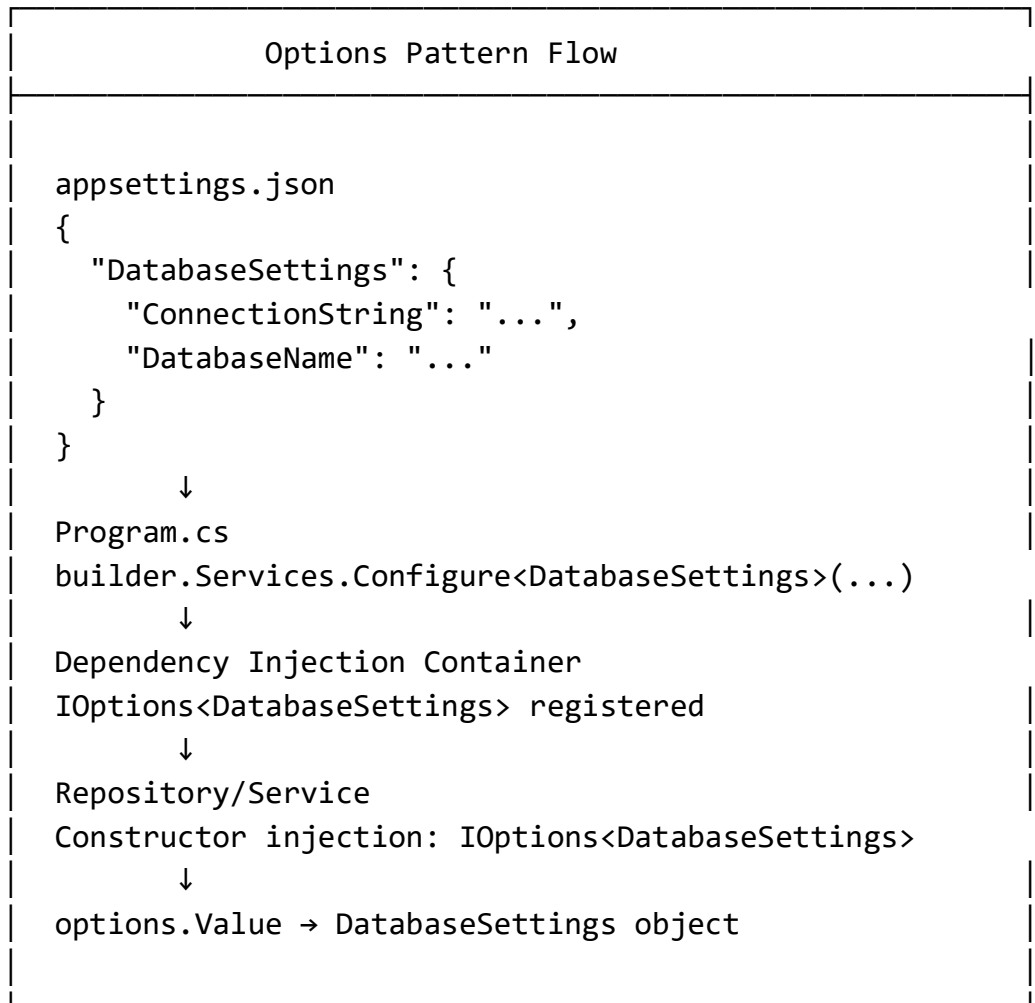
#### **Configuration in Program.cs:**

```
builder.Services.Configure<DatabaseSettings>(
    builder.Configuration.GetSection("DatabaseSettings"));
```

#### **Usage in Repository (DiscountRepository.cs):**

```
public DiscountRepository(IOptions<DatabaseSettings> databaseSettings)
{
    // Uses databaseSettings.Value
}
```

## Part 3: How It Works



## Part 4: Benefits You Get

### 1. Type Safety:

- Compile-time checking

- IntelliSense support
- No string-based configuration access

## 2. Validation:

- Can add validation attributes to settings classes
- Fail fast if configuration is invalid

## 3. Testability:

- Easy to mock in unit tests
- Can inject test configuration

## 4. Maintainability:

- Centralized configuration classes
- Easy to add new settings
- Self-documenting code

# Part 5: Summary

## Your Options Pattern Usage:

| Service         | Settings Class   | Configuration                                              | Usage               |
|-----------------|------------------|------------------------------------------------------------|---------------------|
| <b>Catalog</b>  | DatabaseSettings | MongoDB connection string, database name, collection names | Repositories        |
| <b>Basket</b>   | CacheSettings    | Redis connection string                                    | Redis configuration |
| <b>Basket</b>   | GrpcSettings     | Discount service URL                                       | gRPC client         |
| <b>Discount</b> | DatabaseSettings | PostgreSQL connection string                               | Repository          |

## You're using the Options pattern correctly with:

-  Strongly-typed settings classes
-  Configure<T> in Program.cs
-  IOptions<T> injection in constructors

-  Clean separation of configuration logic

This is the recommended Microsoft pattern for configuration in .NET applications.